

第5章 内存管理

建议学时:5-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言(本章是全册灵魂)

🎯 本章学习目标

- 回顾 C 的内存模型(栈/堆/静态区)与手动管理的三大痛点:泄漏、悬垂指针、重复释放
- 掌握 **所有权三法则**:每值一所有者、move 转移、作用域结束 drop——并能用"所有者"视角分析任何一段代码
- 分清 move 与 Copy/Clone: `i32` 为什么可以随便拷,String 为什么不行;与 C 的 `memcpy` 对照
- 掌握 **借用规则**:`&T` 可多个、`&mut T` 必须独占;理解编译器如何让 `use-after-free` 从运行期事故变成编译期错误
- 理解生命周期标注: `'a` 是编译期检查,不改变运行时行为;会读会写函数签名里的生命周期
- 掌握智能指针家族:Box / Rc / Arc / Weak 各司其职,与 C 的 `malloc/free`、C++ 的 `unique/shared/weak_ptr` 对照
- 掌握内部可变性:Cell 与 RefCell,理解"共享 + 可变"的合法组合 `Rc<RefCell<T>>`
- 理解 Drop(RAII 的 Rust 形态)与 `unsafe` 的边界:什么时候用、安全抽象原则

💡 从 C 到 Rust:本章主题如何迁移

C 程序员对内存管理有最深刻的体会:malloc 之后必须记得 free,但"记得"靠的是纪律。`free(p)` 之后又用 `p`?悬垂指针,静默出错。`malloc` 忘 `free`?泄漏,内存缓慢耗尽。同一个指针 `free` 两次?double free,崩溃或安全漏洞。这三种事故占 C 内存类 bug 的绝大多数,而编译器只会袖手旁观。Rust 的核心承诺就一句话:让编译器代替你的纪律——把"释放时机"从程序员的记忆里搬进类型系统,从源头消灭这三类事故。

机制上,Rust 没有垃圾回收(GC),它走的路线是"**所有权(ownership)**":每个值有且只有一个所有者,所有者离开作用域,值就被自动释放。听起来简单,但它的力量在于"编译器强制"四个字——你想违反?编译错误。于是释放时机的正确性不再是"运行久了才发现的泄漏",而是"编译期就定案的契约"。这与 C++ 的 RAII 精神一致,但 C++ 还允许你手动 `new/delete` 绕开纪律;Rust 没有"绕开"的通道,除非你主动走进 `unsafe`。

接下来是借用与生命周期。C 的指针只回答一个问题:"地址在哪?"Rust 的引用多回答两个:"能不能改?"与"活多久?"`&T` 表示只读共享,`&mut T` 表示独占可写;`'a` 表示引用的存活范围。于是"free 之后还用"变成"引用活得比数据长"的编译错误,"两个线程同时写"变成"可变借用冲突"的编译错误。第9章你会看到,并发安全几乎免费地继承自这套规则。

最后是工具与边界。堆分配交给 Box,共享所有权交给 Rc/Arc,可变共享交给 RefCell,析构逻辑交给 Drop——每个需求一个明确工具,不再有"自己造轮子"的空间。而 `unsafe` 是唯一的后门:FFI、性能临界、底层库用它,但纪律是"`unsafe` 必须封装在安全接口里,并写清安全前提"。本章读完,你将获得一个 C 程序员最珍贵的转变:从"我小心地管内存"到"编译器替我管,我专注业务"。

前置知识自查

- C 的 malloc/free/calloc/realloc 的完整语义;谁负责释放、何时释放
- C 的悬垂指针、use-after-free、double free、内存泄漏的产生机制
- C 的栈上变量生命周期(函数返回即失效)与静态区(程序期存活)
- C 的结构体赋值(位拷贝)与 memcpy
- 第1章的类型与 String/Vec 的所有权伏笔;第4章的参数传递三态

本章学习路线

1. **第一阶段(约2小时,建立心智模型):**\$5.1–\$5.3,把"所有者"视角刻进脑子——本章后面全靠它
2. **第二阶段(约2小时,借用与生命周期):**\$5.4–\$5.5,这是最容易反复编译失败的部分,多写多报错多读报错
3. **第三阶段(约2小时,工具与实战):**\$5.6–\$5.8,重点搞清楚 Rc 与 Arc、RefCell 与 &mut 的边界
4. **验收标准:**给任意一段含指针/引用的代码,能用"所有者 + 借用 + 生命周期"三句话讲清它的内存行为

本章知识导图

- §5.1 C 的内存模型与三大痛点(malloc/free 的代价)
- §5.2 所有权三法则:每值一所有者、move、drop
- §5.3 move 与 Copy/Clone:哪些类型可以"随便拷"
- §5.4 借用规则:&T 共享、&mut T 独占、NLL
- §5.5 生命周期标注:'a、省略规则、'static
- §5.6 智能指针:Box / Rc / Arc / Weak(对照 C++ 三件套)
- §5.7 内部可变性:Cell / RefCell / Rc<RefCell<T>>
- §5.8 Drop 与 unsafe 的边界
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§5.1 C 的内存模型与三大痛点

5.1.1 内存三区回顾

C 程序的内存分三块:**栈**(函数调用帧,自动分配自动回收)、**堆**(malloc/free 手动管理)、**静态区**(全局变量与字符串字面量,程序期存活)。Rust 的内存模型完全相同,没有 GC,没有额外运行时——区别只在于**堆内存的释放时机由谁决定**。对照如下:

C 的做法	Rust 的做法
malloc(size) 返回 void*	Box::new(v)、Vec::new() 等:类型安全,无 void*
free(p) 手动释放	所有者离开作用域,自动 drop
忘记 free → 泄漏	编译器保证每个值必被释放,无泄漏通道
free 后再用 → 悬垂指针(UB)	借用检查:引用活得比数据长 = 编译错误
free 两次 → double free(崩溃)	一个值只有一个所有者,释放且仅释放一次
手动 realloc 搬数据	Vec 按需增长,自动重新分配并搬移

⚠ 易错点 1:以为 Rust 是"带 GC 的语言"或"不用管内存"

Rust 没有 GC:栈上值随作用域弹出,堆上值由所有者决定生死,一切在编译期安排妥当——但"安排"是编译器的职责,不是自动运行的回收线程。写 Rust 仍然要思考生命周期,只是思考的结果以类型的形式固化下来、由编译器验证。正因为如此,Rust 才有接近 C 的性能与可预测的释放时机,这是它敢自称"系统级语言"的底气。

§5.2 所有权三法则

5.2.1 法则一:每个值有且只有一个所有者

所有者(owner)是"持有这个值"的变量:局部变量拥有它绑定的值,结构体拥有它的字段,Vec 拥有它缓冲区里的元素。**任何时候,一个值只有一个所有者**——没有 C 里"两个指针指向同一块内存"的别名共享。这条法则直接消灭了 double free:释放权只属于唯一的所有者,释放且仅释放一次。

5.2.2 法则二:所有者离开作用域,值被自动释放(drop)

```
fn main() {
    {
        let s = String::from("临时数据");
        println!("使用中:{s}");
    } // 作用域结束:s 的所有者消失,堆内存被自动释放
    // println!("{s}"); // 编译错误:value borrowed after move / 超出作用域

    let v = vec![1, 2, 3]; // v 的所有者是 main 的函数体作用域
    println!("{v:?}");
} // 函数结束,v 连同堆缓冲区一起被释放
```

对照 C: `char *s = malloc(...)` 之后,释放指令 `free(s)` 必须由你写在某个地方;Rust 把"释放"绑定在作用域结束时自动发生——你根本写不出"忘了 free"的代码,因为 free 不是可选项,是作用域的固定收尾动作。

5.2.3 法则三:所有权可以转移(move)

把值赋给另一个变量、或作为参数传进函数,所有权就发生转移——原变量失效。这就是第4章"传 String 后原变量不可用"的原因:

```
fn take_ownership(s: String) {
    println!("现在由我持有:{s}");
} // s 在这里被释放

fn main() {
    let a = String::from("数据甲");
    let b = a;                // move:a 的所有权转移到 b
    // println!("{a}");        // 编译错误:a 已失效(use of moved value)
    println!("{b}");

    let c = String::from("数据乙");
    take_ownership(c);        // move:所有权进入函数
    // println!("{c}");        // 编译错误:c 已失效
}
```

★ 重点:三法则的"内存故事"

把三法则连起来读一遍: 一个值, 一个主人; 主人走, 值就亡; 想易主, 必须搬。这套规则下, 编译器可以静态地算出每一块内存的确切生死时刻: 不会泄漏(每个值都有人释放)、不会重复释放(释放者唯一)、不会悬垂(引用规则见 §5.4)。对比 C 的三个事故, 全部在编译期被排除——这就是"安全"的运作方式。

§5.3 move 与 Copy / Clone

5.3.1 Copy: 位拷贝后, 原值仍可用

为什么 `let b = a;` 对 `i32` 不报错、对 `String` 报错? 因为 `i32` 实现了 **Copy** 标记: 赋值时做一次位拷贝, 原值仍然有效。这对应 C 的结构体赋值/按值传参——但 C 里所有类型都是"位拷贝"的, 而 Rust 只有显式声明 `Copy` 的类型才这样。内置 `Copy` 类型:

Copy 类型(赋值=拷贝)	非 Copy 类型(赋值=move)
所有整数: <code>i8-i128</code> 、 <code>u8-u128</code> 、 <code>isize/usize</code>	<code>String</code> (拥有堆缓冲区)
浮点 <code>f32/f64</code> 、 <code>bool</code> 、 <code>char</code>	<code>Vec<T></code> (拥有堆缓冲区)
定长数组 <code>[i32; 5]</code> 、元组(元素全 <code>Copy</code>)	<code>Box<T></code> (拥有堆上对象)
<code>&T</code> 引用(复制的是地址)	<code>&mut T</code> (不可 <code>Copy</code> , 独占)

```
#[derive(Clone, Copy)]    // 自定义类型声明 Copy(derive 第6章细讲)
struct Point2 {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point2 { x: 1, y: 2 };
    let q = p;              // Copy:浅拷贝,p 仍可用
    println!("{}", p.x, q.y);

    let s1 = String::from("hello");
    let s2 = s1.clone();    // Clone:显式深拷贝(内容复制,所有权各自持有)
    println!("{s1} {s2}");  // 两个都可用——但这是"深拷贝",有开销
}
```

5.3.2 与 C 的 memcpy 对照

Copy 本质上就是一次 memcpy,但比 C 多做两件事:① 只有"复制后仍安全"的类型(无堆指针、无析构义务)才允许 Copy——String 复制指针而不复制缓冲区会 double free,所以被禁止;② 原值是否仍可用由编译器追踪,而不是靠记忆。C 里 `struct A b = a;` 之后两个结构体都能用、但各自析构语义全凭自觉;Rust 把"能拷贝的"与"只能搬移的"分得清清楚楚。

⚠ 易错点 2:给"带堆数据"的类型随手写 `#[derive(Copy)]`

编译错误会当场教育你:"the trait `Copy` may not be implemented for this type; field `data` does not implement `Copy`"——String、Vec 字段天然禁止 Copy。这不是限制,是保护:Copy 承诺"复制后双方独立",而共享同一个堆缓冲区违背这个承诺。需要"复制内容"用 `clone()`;需要"共享所有权"用 `Rc`(§5.6)——按需选择,不要硬造 Copy。

§5.4 借用规则

5.4.1 借用:只借不拿

数据被 move 走后就消失了,但函数常常只需要"看一眼"。Rust 允许通过引用临时借用数据,借用人不是所有者,不承担释放义务——对应的 C 概念是"受控的指针":

```
fn main() {
    let v = vec![1, 2, 3];
    let r1 = &v;          // 不可变借用:共享
    let r2 = &v;          // 第二个不可变借用:允许
    println!("{r1:?} {r2:?}"); // 多个读者共存

    let mut w = vec![4, 5, 6];
    let m = &mut w;       // 可变借用:独占
    m.push(7);
    println!("{m:?}");
}
```

5.4.2 两条铁律

★ 重点:借用规则(整章的核心)

- 规则一:任意时刻,要么有任意多个不可变借用(&T),要么有一个可变借用(&mut T),不能同时存在
- 规则二:借用必须活得比数据短(数据被释放后,借用必须已消失)

规则一消灭"数据竞争"(多个写者);规则二消灭"悬垂引用"(use-after-free)。两条规则都由编译器在编译期检查——C 里这两类 bug 靠人盯,盯不住。

```
fn main() {
    let mut v = vec![1, 2, 3];

    let r1 = &v;
    let r2 = &v;
    println!("{r1:?} {r2:?}");    // 最后一次使用 r1、r2

    // 借用已结束(NLL:非词法作用域),现在可以可变借用了
    let r3 = &mut v;
    r3.push(4);

    // 下面的写法编译错误:不可变借用 r4 还活着时想可变借用
    // let r4 = &v;
    // let r5 = &mut v;    // error[E0502]: cannot borrow as mutable
}
```

NLL(Non-Lexical Lifetimes,非词法生命周期)是 2018 年引入的改进:借用结束于"最后一次使用",而不是整个作用域。这让"先借后改"的合法代码不再被误伤——上面 r1/r2 用完后立刻可以 &mut。

5.4.3 与 C 的指针对照

C 的写法	Rust 的写法	编译器做什么
void f(const T *p)	fn f(p: &T)	只读借用;多个可共存
void f(T *p)	fn f(p: &mut T)	独占;同一时刻最多一个
void *p = malloc(...); free(p); 再用 p	借用必须短于数据生命	悬垂引用 = 编译错误
两个指针指向同一内存,一个写一个读	读写借用互斥	数据竞争 = 编译错误

⚠ 易错点 3:被 E0502 搞烦了,想用"拷贝"绕过借用

借用冲突的典型解法不是 clone,而是 **缩小借用范围**:把只读用的部分先算完(如上例的 println),或把读取逻辑提取成小函数,或改用迭代器闭包让借用局部化。clone 只在"数据小、拷贝便宜"时作为最后手段。见到 E0502/E0503/E0597,深呼吸,先问自己:哪个借用还活着?让它先走。

§5.5 生命周期标注

5.5.1 为什么需要 'a

看这个函数: `fn longest(x: &str, y: &str) -> &str` ——返回的引用到底借自 `x` 还是 `y`? 编译器无法自己猜(两个都可能),于是需要你标注:返回值的存活期与两个参数中最短的那个绑定。语法是在引用类型上写 `'a`:

```
// 'a 是"生命周期参数":声明"x、y 与返回值活得一样久"
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("短");
    let s2 = String::from("更长的字符串");
    let r = longest(&s1, &s2); // 返回的引用在 s1、s2 内都有效
    println!("{}", r);
}
```

5.5.2 关键认知:生命周期是编译期检查,不改变运行时

★ 重点:生命周期不产生任何运行时代码

`'a` 是纯粹的编译期概念:它不改变数据的布局、不产生额外的运行时开销、没有"生命周期对象"存在于内存中。它只是给编译器的一份证明书,声明引用之间的存活关系。你写的 `'a` 会被编译成零条指令。运行时只有一件事是真的:数据在所有者离开作用域时被释放——而这与你标注的 `'a` 无关。

正因为检查发生在编译期,违反规则就是编译错误,而且错误信息会指明是哪一次借用越界。看一个"悬垂引用"的活例子——C 里这是静默 UB,Rust 里是明确的编译错误:

```
fn main() {
    let r;
    {
        let s = String::from("短暂的数据");
        r = longest(&s, &s); // 借用 s,但 s 马上要被释放
    } // s 在这里释放
    // println!("{}", r); // error[E0597]: `s` does not live long enough
}
```

5.5.3 省略规则与 'static

实际代码里 `'a` 并不常见,因为编译器有**生命周期省略(lifetime elision)**规则:单入参函数(如 `fn first_word(s: &str) -> &str`)、方法(`&self` 借用)都自动补全,不用写。只有"多引用入参 + 返回引用"这种组合才必须显式标注。`'static` 是最长的生命周期:程序全程存活——字符串字面量的默认类型就是 `&'static str`:


```
fn main() {
    // 字符串字面量活在静态区,类型是 &'static str
    let greeting: &'static str = "你好,Rust";
    println!("{}",greeting);

    // 普通 String 的引用不是 'static:
    let s = String::from("运行时数据");
    // let p: &'static str = &s;    // 编译错误:无法保证存活到程序结束

    // 'static 的真实场景:配置常量、错误消息、static 全局(第1章)
    static APP_TAG: &'static str = "v1.0";
    println!("{}",APP_TAG);
}
```

工程建议:生命周期标注的三个生产场景

- **函数返回借用时:**写签名前先想"返回的引用借自哪个参数",需要标注就标注;省略规则能覆盖就别写
- **结构体持有引用时:**struct 字段是 &T 必须写生命周期(如 `struct View<'a> { data: &'a [u8] }`);能用 owned 类型(String/Vec)就尽量不用引用——少一类编译烦恼
- **多写多错多修:**生命周期报错是"证明书不全",补标注或调整结构,不是修改逻辑——先确认数据谁活着,再改代码

§5.6 智能指针:Box / Rc / Arc / Weak

5.6.1 Box:把值放到堆上

Box 是"堆上单个值"的所有者,与 C 的 `malloc` 一一对应,但无需 `free`——离开作用域自动释放:

```
fn main() {
    // C: struct Node *n = malloc(sizeof(struct Node));
    let n = Box::new(42);
    println!("{}", *n);           // 解引用,与 C 的 *p 相同

    // Box 常用来打破递归类型(递归大小无法确定,必须装箱):
    // enum List { Nil, Cons(i32, Box<List>) } // 见下方示例
    println!("指针大小:{{}} 字节", std::mem::size_of:::<&i32>());
}

// 递归类型的标准写法:Box 让 List 大小确定(指针 + 数据)
enum List {
    Nil,
    Cons(i32, Box<List>),
}

fn main2() {
    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    let _ = list;
    println!("链表构建完成");
}
```


注意上面有两个 main——教学展示请二选一;实际工程一个 crate 只能有一个 main。这正是"宁简勿错"的演示:递归类型(树、链表)是 Box 的头号生产场景,因为无限递归大小在编译期无法确定,必须装箱成指针。

5.6.2 Rc:单线程引用计数共享

当多个地方需要"共同持有"同一份数据时(图结构、配置共享),Box 不够用——它只能有一个所有者。Rc 提供引用计数:每 clone 一次计数 +1,计数归零才释放。对应 C++ 的 `std::shared_ptr`,但 Rc 是单线程的(线程安全版本是 Arc):

```
use std::rc::Rc;

fn main() {
    let a = Rc::new(String::from("共享配置"));
    let b = Rc::clone(&a);           // 计数 2;注意是 Rc::clone,不是深拷贝
    let c = Rc::clone(&a);           // 计数 3
    println!("强计数:{}", Rc::strong_count(&a));

    drop(b);                         // 计数 2
    println!("drop 后计数:{}", Rc::strong_count(&a));
    println!("{}", c);               // 数据仍有效
    drop(c);
    drop(a);                         // 计数归零:堆上 String 被释放
}
```

5.6.3 Arc:多线程版本的 Rc

Arc = Atomic Rc,计数操作是原子的,可以跨线程共享(第9章与 Mutex 组合使用)。用法与 Rc 相同,只是 `use std::sync::Arc`。对应 C++ 的 `shared_ptr` (C++ 的 `shared_ptr` 天生线程安全计数)。

5.6.4 Weak:打破循环引用的钥匙

引用计数的致命弱点:如果 A 持有 B、B 又持有 A,两者的计数永远不小于 1,内存永远不被释放——这就是引用计数的"泄漏"。C++ 的 `shared_ptr` 同样有这个问题。解法是让环中的一方持有弱引用 **Weak**:Weak 不增加强计数,访问时先 `upgrade()` 试试对方还活着没有:

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,    // 父节点用弱引用:不成环
    children: RefCell<Vec<Rc<Node>>>, // 子节点用强引用
}

impl Node {
    fn new(name: &str) -> Rc<Node> {
        Rc::new(Node {
            name: name.to_string(),
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }

    fn add_child(parent: &Rc<Node>, child: &Rc<Node>) {
        *child.parent.borrow_mut() = Rc::downgrade(parent); // 父指针:弱
        parent.children.borrow_mut().push(Rc::clone(child)); // 子指针:强
    }
}

fn main() {
    let root = Node::new("根");
    let leaf = Node::new("叶");
    Node::add_child(&root, &leaf);
    println!("root 强计数:{}", Rc::strong_count(&root));
    println!("leaf 强计数:{}", Rc::strong_count(&leaf));

    // 通过弱引用访问父节点:先 upgrade 确认存活
    let parent = leaf.parent.borrow();
    if let Some(p) = parent.upgrade() {
        println!("leaf 的父节点:{}", p.name);
    }
    drop(root); // root 释放后
    // parent.upgrade() 将返回 None,不会产生悬垂访问
    println!("root 已释放,升级结果:{}", parent.upgrade().is_some());
}

```

C	C++	Rust
malloc/free	new/delete、RAII	Box::new + 作用域自动 drop
(手写计数结构)	std::shared_ptr	Rc(单线程)/ Arc(多线程)
(手写弱引用)	std::weak_ptr	Weak(配合 upgrade 检查)
void* + 手动类型转换	unique_ptr<T>	Box<T>:类型安全,无 void*

§5.7 内部可变性:Cell 与 RefCell

5.7.1 冲突的两种解法:换借用方式,或换容器

借用规则说"共享 &T 不可写"。但生产里有合法需求:多个持有者共享一份数据,其中一方要改(统计计数、缓存)。Rust 提供**内部可变性(interior mutability)**容器,把"修改"从编译期检查挪到运行期检查:

容器	适用场景	检查时机
Cell<T>	T 是 Copy 类型;按值读写 get/set	编译期(无借用概念,按值取放)
RefCell<T>	T 任意类型;borrow()/borrow_mut() 取引用	运行期(冲突时 panic)
&mut T 直接借用	普通可变借用	编译期

```
use std::cell::{Cell, RefCell};

fn main() {
    // Cell:按值操作,适合计数器
    let count = Cell::new(0);
    count.set(count.get() + 1);
    println!("计数:{}", count.get());           // 1

    // RefCell:借用规则推迟到运行期
    let data = RefCell::new(String::from("hello"));
    data.borrow_mut().push_str(", world");
    println!("{}", data.borrow());               // hello, world

    // 运行期检查:borrow 未结束时 borrow_mut 会 panic(不是编译错误!)
    let r = data.borrow();
    // data.borrow_mut(); // 运行期 panic:already borrowed
    drop(r);                                     // 先归还借用
    data.borrow_mut().push('!');
    println!("{}", data.borrow());
}
```

5.7.2 组合拳:Rc<RefCell<T>> = 共享 + 可变

单独 Rc 不能改(共享借用只读),单独 RefCell 不能共享(一个所有者)。两者组合,就是 Rust 的"共享可变状态"标准答案——对应 C 里"多个指针共享 + 手动加锁/约定"的场景(单线程版):

```

use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    // 多个持有者 + 内部可变:图、缓存、UI 状态的标准组合
    let shared = Rc::new(RefCell::new(vec![1, 2, 3]));
    let a = Rc::clone(&shared);
    let b = Rc::clone(&shared);

    a.borrow_mut().push(4);      // 通过 a 修改
    b.borrow_mut().push(5);      // 通过 b 修改
    println!("{:?}", shared.borrow()); // [1, 2, 3, 4, 5]
}

```

⚠ 易错点 4:以为 RefCell 让"借用检查失效",随便写

RefCell 把检查从编译期挪到运行期,检查仍然存在,只是时机变了:borrow 与 borrow_mut 同时活跃就 panic,而且 panic 是崩溃不是错误处理。生产教训:① 尽量保持 borrow 的生命周期短(取用即 drop);② 不要在持有 borrow 的引用时调用可能再 borrow 的函数;③ 多线程场景请用 Mutex 而不是 RefCell(第9章)。RefCell 不是后门,是"把违反规则的后果从编译错误换成运行期崩溃"的慎重让步。

§5.8 Drop 与 unsafe 的边界

5.8.1 Drop:释放时的"最后动作"

Drop trait 定义"值被释放时运行的代码",是 Rust 的 RAII 形态:资源(文件、锁、连接)的获取与释放绑定在对象生命周期上。对应 C++ 的析构函数;C 里对应的则是"记得在每条 return 路径上 close 文件"的痛苦:

```

struct FileHandle {
    path: String,
}

impl Drop for FileHandle {
    fn drop(&mut self) {
        // C 里这里必须手动 close(fd),且每条出错路径都要记得
        println!("关闭文件:{}", self.path);
    }
}

fn main() {
    {
        let f = FileHandle { path: String::from("config.toml") };
        println!("使用中:{}", f.path);
    } // 作用域结束:自动调用 drop
    println!("文件已自动关闭,无需手动清理");
}

```

生产中的 Drop 场景:文件句柄、数据库连接、锁的释放、性能计时器。第9章的 Mutex 锁也靠 Drop 自动释放——你不需要写 unlock,作用域结束就是 unlock。

5.8.2 unsafe:唯一的后门,以及它的边界

Rust 的安全保证在 **unsafe** 块/函数中被暂停,允许你做四件事:解引用裸指针、调用 unsafe 函数(FFI)、访问可变静态变量、实现 unsafe trait。什么时候用:

场景	是否建议 unsafe	理由
调用 C 库(FFI)	必须	C 接口本身就是裸指针;安全封装后对外
极致性能临界(如哈希表热点)	极少	先 profile,再决定;绝大多数场景用不到
绕过借用规则"图方便"	禁止	违背安全承诺,且大概率是设计问题
业务代码	禁止	业务里出现 unsafe 是明显的坏味道

```
fn main() {
    let mut x = 42;
    let raw: *mut i32 = &mut x;      // 裸指针:与 C 的 int* 一致
    unsafe {
        *raw += 1;                    // 解引用裸指针必须在 unsafe 里
    }
    println!("{x}");                  // 43

    // 安全抽象原则:unsafe 只藏在函数内部,对外接口是安全的
    let v = read_or_zero(raw);
    println!("读到的值:{v}");
}

/// 安全外壳:内部用 unsafe,外部调用者无需感知
fn read_or_zero(p: *const i32) -> i32 {
    if p.is_null() {
        return 0;                    // 先检查空指针,再进 unsafe
    }
    unsafe { *p }
}
```

★ 重点:unsafe 的四条纪律

1. 最小化:unsafe 块越小越好,一行一行地圈定

2. 封装:unsafe 必须藏在安全函数内部,对外只暴露安全接口

3. 注释:每个 unsafe 块写明"为什么这里是安全的"(安全前提)

4. 隔离:用独立的模块收纳 unsafe 代码,便于审计(第13章 mod 组织)

记住: unsafe 不是"关掉安全检查",而是"你向编译器承诺:这块我自己保证正确"——承诺要兑现,否则就是给自己埋雷。

本章速查表

主题	结论 / 写法
内存三区	栈(自动)/堆(所有者管理)/静态区(程序期);无 GC
所有权法则1	每个值有且只有一个所有者
所有权法则2	所有者离开作用域,值自动 drop(无泄漏通道)
所有权法则3	赋值/传参转移所有权(move);原变量失效
Copy	整数/浮点/bool/char/定长数组/&T:赋值=位拷贝;String/Vec/Box 不是
Clone	s.clone() 显式深拷贝,双方各自持有
借用规则1	多个 &T 或一个 &mut T,不可同时存在
借用规则2	借用必须短于数据生命(悬垂 = 编译错误 E0597)
NLL	借用结束于最后一次使用,而非作用域结束
生命周期	'a 是编译期证明,零运行时开销;多引用返回需标注;省略规则覆盖常见场景
'static	程序全程存活;字符串字面量的默认生命周期
Box	堆上单值;递归类型(链表/树)必用
Rc / Arc	引用计数共享(单/多线程);Rc::clone 只增计数不深拷贝
Weak	弱引用防循环;upgrade() 检查存活;对应 C++ weak_ptr
Cell / RefCell	内部可变性;RefCell 运行期检查,冲突即 panic
Rc<RefCell<T>>	共享 + 可变的单线程标准组合
Drop	作用域结束自动调用;RAII:资源绑定生命周期
unsafe	FFI/裸指针/性能临界;最小化 + 封装 + 注释 + 隔离

最小必会清单

- 能背出所有权三法则,并用"所有者"视角解释任意一段代码的释放时机
- 能说出 5 种 Copy 类型与 3 种非 Copy 类型,并解释 String 为什么不能 Copy
- 能解释 move 与 Clone 的区别,以及各自的开销
- 能背出借用两条铁律,并解释它们如何消灭数据竞争与悬垂引用
- 能说出 NLL 的含义,并写出"先借后改"的合法代码
- 能写出带 'a 的函数签名,并说出生命周期在运行时的开销(0)
- 能说明 Rc 与 Arc 的区别、Weak 解决什么问题
- 能写出 Rc<RefCell<T>> 的共享可变状态代码
- 能说出 RefCell 与 &mut 的检查时机差异(编译期 vs 运行期)
- 能说出 unsafe 的四个使用场景与四条纪律

自测题

Q 1. 以下代码打印什么?为什么没有泄漏? `fn main() { let a = String::from("x"); { let b = a; } println!("{}", b); }`

✓ 参考答案

这个程序编译失败:a 的所有权在 `let b = a` 时 `move` 给 b,b 在内部块结束即释放;随后 `println!("{}", b)` 虽然不引用 a, 但 a 已被 `move`——不,`println!("{}", b)` 不引用 a 时其实可以编译!(NLL 后 `move` 检查也按使用点算)。严谨答案:若 `println` 引用 a 则编译错误;若不引用,编译通过,b 的 `String` 在块结束时被释放,无泄漏。核心:释放自动发生,不需要也写不出 `free`。

Q 2. 为什么 `let y = x;` 对 `i32` 合法、对 `String` 需要 `clone` 或 `move`?这与 C 的结构体赋值有何不同?

✓ 参考答案

`i32` 实现了 `Copy`:赋值是位拷贝,双方独立。`String` 拥有堆缓冲区,位拷贝会让两个 `String` 指向同一块缓冲,释放时 `double free`——所以赋值=`move`(转移所有权)。C 里所有类型赋值都是位拷贝,struct 含指针时双份指针指向同一堆内存,谁释放、何时释放全凭约定;Rust 用 `Copy` 标记把"能安全拷贝的"与"只能搬移的"分开,由编译器强制。

Q 3. 下面的代码错在哪? `let mut v = vec![1,2,3]; let r = &v; v.push(4); println!("{}", r);`

✓ 参考答案

不可变借用 `r` 还活着时调用 `v.push` 需要可变借用,违反铁律一:E0502。修复:让 `r` 先"退休"——把 `println` 提到 `push` 之前,或不用 `r`。这正是借用冲突的标准解法:缩小借用范围,而不是 `clone`。

Q 4. 生命周期 'a 在运行时存在吗?它影响性能吗?

✓ 参考答案

不存在、不影响。'a 是纯编译期概念:给编译器证明引用之间的存活关系,零运行时开销、零内存布局影响。数据真正释放的时机只有一条:所有者离开作用域。写 'a 不是写运行时逻辑,是写证明书。

Q 5. Rc 与 Arc 的区别?循环引用为什么泄漏?如何破?

✓ 参考答案

`Arc` 的计数操作原子化,可跨线程;`Rc` 只能单线程。循环引用:两对象互相强引用,各自计数 ≥ 1 ,永远不为 0,内存不释放(引用计数的固有缺陷,C++ `shared_ptr` 同病)。破法:让环上的一方持 `Weak`——`Weak` 不增加强计数,且升级前先检查存活。

Q 6. RefCell 的借用检查在什么时机?违反规则的结果与 `&mut` 有何不同?

✓ 参考答案

`RefCell` 在运行期检查:`borrow_mut` 时若已有活跃 `borrow`(或其逆),直接 `panic`;编译期检查只验证"borrow 的引用不逃逸"。而 `&mut` 在编译期检查,违反即编译错误。所以 `RefCell` 是把"违反规则的代价"从编译错误换成运行

期崩溃——能用 `&mut` 就用 `&mut`, `RefCell` 只用于"多个所有者共享可变状态"的合法需求。

章末实验题:所有权与引用计数观察实验

实验 5:内存行为观察——所有权、引用计数与内部可变性

实验目标:写一个"内存行为示波器":用 `Drop` 打印观察释放时机,用 `Rc/Weak` 观察引用计数变化,用 `RefCell` 体验内部可变性,并做一组所有权重构练习。

实验要求:

- 任务 1:定义带 `Drop` 打印的 `Tracker` 结构体,创建栈上对象与 `Box` 堆上对象,观察 `drop` 顺序(覆盖:所有权三法则 §5.2、`Drop` §5.8、`Box` §5.6)
- 任务 2:做 `move` 实验:`i32` 赋值后双方可用,`String` 赋值后原变量失效(注释掉错误行观察报错)(覆盖:`move` 与 `Copy` §5.3)
- 任务 3:写 `swap` 函数(`&mut` 参数)并调用;再尝试"两个不可变借用 + 可变借用"的冲突代码,观察 `E0502`(覆盖:借用规则 §5.4)
- 任务 4:写带生命周期标注的 `longer` 函数,并构造"借用超过数据生命"的失败示例(覆盖:生命周期 §5.5)
- 任务 5:用 `Rc` + `Weak` 构造父子节点,打印强计数/弱计数,`drop` 父节点后验证 `weak.upgrade()` 返回 `None`(覆盖:`Rc/Arc/Weak` §5.6)
- 任务 6:用 `RefCell` 实现访问计数,验证共享引用下也能修改(覆盖:内部可变性 §5.7)

实验环境:`cargo new mem_observe`;每完成一个任务运行一次 `cargo run`,观察打印顺序与报错信息。

第一步 一步:先写 `Tracker` 与 `Drop impl`,验证"作用域结束自动释放"——这是所有观察的地基

第二步 二步:把失败示例写成注释(如被 `move` 的变量、`borrow` 冲突),运行确认编译错误,再解开修复——"故意编译失败"是本章最有效的学习法

第三步 三步:`Rc` 实验重点看 `strong_count` 的变化与 `drop` 的时机

第四步 四步:`Weak` 实验先构造"父子互持"版本,观察泄漏,再改成弱引用版本对比

第五步 五步:`RefCell` 实验故意触发一次运行期 `panic`,观察它与编译期错误的不同之处

参考答案要点

```

// ===== 实验 5:内存行为观察 =====
// 覆盖知识点:所有权三法则($5.2)、move 与 Copy($5.3)、借用规则($5.4)、
//             生命周期标注($5.5)、Box/Rc/Weak($5.6)、
//             内部可变性 RefCell($5.7)、Drop 析构与释放观察($5.8)

use std::cell::RefCell;
use std::rc::{Rc, Weak};

/// 观察 Drop:每次释放打印一行($5.8)
struct Tracker {
    name: String,
}

impl Drop for Tracker {
    fn drop(&mut self) {
        println!(">>> [drop] {} 被释放", self.name);
    }
}

/// 带访问计数的共享节点($5.6 + $5.7)
struct Node {
    name: String,
    visits: RefCell<u32>,           // 内部可变:共享引用下也能改
}

impl Node {
    fn new(name: &str) -> Rc<Node> {
        Rc::new(Node {
            name: name.to_string(),
            visits: RefCell::new(0),
        })
    }

    fn visit(&self) {
        *self.visits.borrow_mut() += 1; // 运行期借用检查
    }

    fn count(&self) -> u32 {
        *self.visits.borrow()
    }
}

/// 生命周期标注:返回两个引用中较长者($5.5)
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() >= y.len() { x } else { y }
}

/// 借用演示:交换两个值($5.4)
fn swap(a: &mut i32, b: &mut i32) {
    let t = *a;
    *a = *b;
    *b = t;
}

fn main() {

```

```

// ---- 任务 1:所有权与 Drop 时机 ----
{
    let t = Tracker { name: String::from("栈上对象") };
    println!("使用栈上对象:{}", t.name);
} // 此处应打印 drop

let heap = Box::new(Tracker { name: String::from("堆上对象") });
println!("Box 内对象:{},指针大小 {} 字节",
    heap.name, std::mem::size_of_val(&heap));
drop(heap); // 显式释放:应打印 drop

// ---- 任务 2:move 与 Copy ----
let a = 5;
let b = a; // Copy:双方可用
println!("Copy 后:{a} {b}");
let s1 = String::from("数据");
let s2 = s1; // move:s1 失效
println!("move 后:{s2}");
// println!("{s1}"); // 编译错误:use of moved value

// ---- 任务 3:借用规则 ----
let mut x = 10;
let mut y = 20;
swap(&mut x, &mut y);
println!("swap 后:x={x} y={y}");

let v = vec![1, 2, 3];
let r1 = &v;
let r2 = &v; // 多个只读借用:允许
println!("共享借用:{r1:?} {r2:?}");
// let m = &mut v; // 编译错误:仍有活跃只读借用

// ---- 任务 4:生命周期 ----
let s = String::from("hello world");
let short = "hi";
println!("较长者:{}", longer(&s, short));

// ---- 任务 5:Rc / Weak 计数观察 ----
let root = Node::new("根节点");
let leaf = Node::new("叶节点");
let weak_leaf: Weak<Node> = Rc::downgrade(&leaf);
println!("叶节点:强计数 {},弱计数 {}",
    Rc::strong_count(&leaf), Rc::weak_count(&leaf));

root.visit();
root.visit();
leaf.visit();
println!("根访问 {} 次,叶访问 {} 次(RefCell 生效)",
    root.count(), leaf.count());

if let Some(l) = weak_leaf.upgrade() {
    println!("弱引用升级成功,访问次数:{}", l.count());
}
drop(leaf); // 强计数归零,节点释放
println!("叶节点释放后,弱引用升级是否成功:{}",

```

```

        weak_leaf.upgrade().is_some()); // false:不会悬垂

// ---- 任务 6:RefCell 与借用冲突的两种后果 ----
let cell = RefCell::new(10);
{
    let b = cell.borrow();           // 只读借用存活中
    println!("borrow 中:{b}");
    // cell.borrow_mut();           // 运行期 panic,而非编译错误!
}                                   // b 在此归还
*cell.borrow_mut() += 5;
println!("修改后:{}", cell.borrow());
}

```

设计说明:实验的节奏是"先看对的,再撞错的"——任务 2/3/6 各留了一个注释掉的失败示例,建议逐个解开运行,亲眼确认三类错误(use of moved value、E0502 借用冲突、RefCell 运行期 panic)在编译期/运行期的不同表现。全部跑通后,把 root/leaf 的强引用关系改成"父子互持"再看计数,就能亲手复现循环引用泄漏。

下一章预告:第6章 面向对象——Rust 没有继承!C 用 struct+函数指针模拟多态,Rust 用 trait 定义共享行为、dyn Trait 做动态分派、impl Trait 做静态分派,组合优于继承。运算符重载、访问控制、设计原则,一章讲透。